

T01-System & OS internals

1. Computer Architecture: Overview and Key Concepts

Computer architecture defines the structural and behavioral blueprint of a computing system, encompassing the Instruction Set Architecture (ISA), microarchitecture, and system-level organization. It dictates measurable system properties such as performance, scalability, reliability, and energy efficiency.

- **Fundamental Models:** The traditional Von Neumann model utilizes unified memory for instructions and data but suffers from the Von Neumann bottleneck. Alternatives include Harvard-style split memory and domain-specific architectures.
- **Core Components:** ISAs (x86, ARM, RISC-V) define programmer-visible operations; microarchitecture dictates hardware implementation (pipelines, caches, predictors); system-level organization manages memory hierarchies and interconnects.
- **Performance Techniques:** Include pipelining, superscalar/out-of-order execution, branch prediction, caching, and parallelism (multicore, SIMD, accelerators).
- **Key Limitations & Trends:** The end of Dennard scaling and slowing of Moore's Law have driven a shift toward heterogeneous architectures (CPU + GPU + AI accelerators), open ISAs (RISC-V), chiplet designs, 3D stacking/HBM, and security-aware hardware post-Spectre/Meltdown.
- **Open Problems:** Energy-efficient scaling, memory wall mitigation (processing-in-memory), and formal hardware verification.

2. CPU Fundamentals: Structure, Operation, and Advanced Features

The Central Processing Unit (CPU) is the core computational engine, consisting of the ALU/FPU, control unit, and registers.

- **Execution Model:** Modern CPUs operate on a Fetch-Decode-Execute cycle, often decomposing complex CISC instructions into internal RISC-like micro-ops.

- **Advanced Features:** Performance is maximized via pipelining, superscalar execution (multiple issues per cycle), out-of-order execution, speculative execution, and simultaneous multithreading (SMT).
- **Design Philosophies:** RISC prioritizes simple, pipelined instructions, while CISC relies on internal translation to micro-ops. Modern mobile/SoC CPUs utilize hybrid designs (big.LITTLE) to balance performance and energy efficiency.
- **Benchmarks & Metrics:** Performance is measured via IPC (Instructions Per Cycle), clock frequency, and benchmarks like SPEC CPU and MLPerf. Apple's M-series chips exemplify recent leaps in performance-per-watt.
- **Open Problems:** Overcoming the memory wall, securing microarchitectures against side-channels, and scaling many-core cache coherency.

3. Memory Fundamentals: Hierarchy, Types, and System Implications

Memory systems exploit locality to bridge the speed gap between fast CPUs and slow persistent storage.

- **Hierarchy:** Registers → L1/L2/L3 Caches (SRAM) → Main Memory (DRAM) → Persistent Storage.
- **Virtual Memory:** The OS manages isolated address spaces via paging, accelerated by TLBs, extending physical memory to disk at high latency costs.
- **Coherence & Consistency:** Multi-core systems rely on protocols like MESI for cache coherence. Consistency models (strong in x86 vs. weak in ARM/RISC-V) dictate software visibility of memory operations.
- **Security & Reliability:** Threats include Rowhammer and cache side-channels. ECC memory mitigates bit flips, while emerging persistent memory (NVM/Optane) requires crash-consistent programming.
- **Open Problems:** Processing-in-memory (PIM) to reduce data movement, and developing scalable coherence for many-core processors.

4. Storage Systems: Media, Interfaces, and System Designs

Storage provides persistent data retention, evolving rapidly from mechanical disks to advanced flash and memory-class technologies.

- **Media Types:** HDDs offer high capacity at low cost but suffer high latency. SSDs (NAND flash) provide low latency and high IOPS, though wear-leveling and write amplification remain concerns. Emerging storage-class memory (Optane) bridges the DRAM-SSD gap.
- **Interfaces:** NVMe over PCIe has revolutionized storage by reducing CPU overhead and increasing parallelism compared to SATA.
- **System Topologies:** Ranging from local DAS to networked NAS/SAN and massive-scale object storage (S3). Distributed file systems (HDFS, Ceph) trade strict consistency for scalability.
- **Reliability:** RAID and erasure coding provide redundancy, while advanced filesystems combat silent corruption and bit rot via checksums and scrubbing.
- **Trends:** Disaggregated storage, computational storage (offloading to SSD controllers), and NVMe-oF for low-latency remote storage.

5. Operating Systems: Functions, Architectures, and Modern Trends

The OS abstracts hardware and manages resources, providing essential APIs and system calls for applications.

- **Architectures:** Monolithic kernels (Linux, Windows) include all services in kernel space for performance. Microkernels (Minix, QNX) run minimal services in kernel space for reliability. Hybrid kernels (macOS XNU) blend both.
- **Core Subsystems:** Process/thread scheduling, virtual memory management, I/O/device drivers, and security enforcement (SELinux, capabilities).
- **Modern Trends:** OS-level virtualization support (hypervisors, containers), power-aware scheduling, kernel bypass (DPDK, SPDK) for low latency, and eBPF for advanced observability.
- **Safety & Scalability:** Experiments with memory-safe languages (Rust in Linux) and formal verification (seL4) are addressing kernel vulnerabilities and hardening OS foundations.
- **Open Problems:** Scaling to many cores, memory safety in kernel code, and providing deterministic real-time guarantees in general-purpose OSes.

6. Processes and Threads: Concurrency, Synchronization, and Scheduling

Processes and threads are the fundamental units of execution and concurrency managed by the OS.

- **Definitions:** A process is an isolated execution environment with dedicated resources; a thread is an execution context within a process that shares its resources.
- **Synchronization:** To prevent race conditions, systems use mutexes, semaphores, and condition variables. Highly concurrent systems employ futexes (efficient user/kernel mutexes) and lock-free/wait-free algorithms.
- **IPC & Parallel Models:** Threads/Processes communicate via pipes, sockets, shared memory, and message queues. Modern programming models range from pthreads/OpenMP to actor models (Erlang) and ownership-based safety (Rust).
- **Common Pitfalls:** Concurrency introduces deadlocks, livelocks, and starvation, which require specialized debugging tools (thread sanitizers, race detectors).
- **Open Problems:** Deterministic concurrency models, scalable schedulers for manycore systems, and automatic parallelization.

7. File Systems: Organization, Features, and Advanced Designs

File systems organize raw storage blocks into accessible files and directories, managing metadata, permissions, and data integrity.

- **Key Types:** Journaling filesystems (ext4, NTFS) expedite crash recovery. Copy-on-Write (ZFS, Btrfs) enables atomic updates, snapshots, and cloning. Log-structured/flash-optimized filesystems (F2FS) reduce SSD write amplification.
- **Features:** Modern filesystems emphasize data integrity (checksums), security (encryption/ACLs), and failure recovery (self-healing/scrubbing).
- **Distributed File Systems:** Systems like HDFS trade strict POSIX semantics for massive scalability and fault tolerance via replication.
- **Open Problems:** Namespace management for exabyte-scale storage, immutable stores for ransomware defense, and stronger distributed semantics.

8. System Boot Process: Firmware to Userland

The boot process transitions a system from a powered-off state to an operational OS, establishing a chain of trust.

- Phases: POST (hardware checks) → Firmware (device init) → Bootloader (loads kernel) → Kernel Initialization (drivers, root mount) → User-space Startup (systemd/init).
- Legacy vs. Modern: Legacy BIOS + MBR relies on 512-byte bootstrap code. Modern UEFI + GPT supports large disks, richer pre-boot environments, and Secure Boot.
- Security: Secure Boot validates cryptographic signatures to prevent rootkits. Trusted Platform Modules (TPM) enable measured boot for system attestation.
- Advanced Boot: Network boot (PXE), virtualized boot (OVMF), and fast-boot techniques (kexec, snapshot resume).
- Open Problems: Firmware hardening against supply-chain attacks (e.g., LoJax rootkits) and improving boot speed for IoT/embedded devices.

9. Applications and Services: Relationship to OS and System Design

Applications and services represent the user-facing and background workloads that rely on the underlying OS and hardware.

- Distinctions: Applications feature user interfaces (browsers, editors), while services/daemons run in the background (databases, web servers).
- Architecture Evolution: Moving from monolithic apps to distributed microservices (orchestrated by Kubernetes) and serverless/FaaS models for ephemeral, event-driven compute.
- System Dependencies: Performance relies heavily on OS configuration (I/O schedulers, CPU topologies) and hardware (GPU acceleration, crypto offload).
- Operations: Modern designs emphasize observability (tracing, metrics), fault isolation (circuit breakers), and security (mTLS, zero-trust).
- Open Problems: Multi-tenant performance isolation, autoscaling algorithms, and secure service communication.

10. Virtualization Basics: Concepts, Types, and System Impacts

Virtualization decouples software from physical hardware, allowing multiple isolated workloads to run on a single host.

- **Hypervisors:** Type 1 (bare-metal: ESXi, Hyper-V) runs directly on hardware for maximum performance. Type 2 (hosted: VirtualBox) runs atop a host OS for ease of use.
- **Hardware Assist & Paravirtualization:** CPU extensions (Intel VT-x, AMD-V) reduce virtualization overhead. Paravirtualized drivers (virtio) improve I/O efficiency via host-guest cooperation.
- **Containers vs. VMs:** Containers (Docker) virtualize the OS level (sharing the kernel via namespaces/cgroups) for lightweight, fast startup. VMs provide stronger isolation with separate kernels at higher overhead.
- **Cloud Impact:** Virtualization is the backbone of multi-tenant clouds, enabling consolidation, portability, live migration, and software-defined storage/networking.
- **Security & Overhead:** Threats include VM escape and shared hardware side-channels, mitigated by IOMMU and strict isolation. Overhead is reduced via SR-IOV, CPU pinning, and hugepages.
- **Open Problems:** Lightweight virtualization (unikernels), secure enclaves (SGX, SEV), and performance predictability for co-located workloads.

Conclusion

Modern computer systems represent a tightly integrated stack where advancements at one layer necessitate adaptations at another. The physical limitations of scaling transistors (Dennard scaling, Moore's Law) have shifted the paradigm from generic, frequency-driven performance to heterogeneous, domain-specific, and heavily parallelized architectures. Concurrently, the evolution from bare-metal monoliths to distributed microservices and virtualized/cloud-native environments has placed immense pressure on OS schedulers, memory hierarchies, and storage fabrics. Moving forward, the most pressing challenges—and research opportunities—lie in mitigating the memory wall, ensuring robust security (both in hardware and software), and achieving energy-efficient scalability across increasingly complex systems.